

Lab #4

Group number : 5

Group members : Aditya Ray - 1233750715

Krutyanjay Shinde - 1234108189

Pre - lab

1. In this lab, you will write an interrupt service routine (ISR) that responds when you push button A on the Pololu robot. The NVIC on the RP2040 can be set to request an interrupt when the voltage level on a GPIO pin is high, when it is low, when it transitions from high to low (falling edge), or when it transitions from low to high (rising edge). If you would like your ISR to respond exactly once to pressing the B button, and you would like it to respond as quickly as possible, which of these four options should you choose? What do you think will happen if you choose the other options?

Hint: Refer to the schematic in the previous lab showing GPIO pin 25 and button A.

- To get the fastest response, we need to set the interrupt trigger to be on the falling edge. GPIO 25 is set to pull-up using resistor R13, and when we press button A, we will get a falling edge first.
 - If we choose to set the interrupt to be low level-triggered, the interrupt would trigger again if the user does not release the button before the interrupt routine is over.
 - If we set the interrupt to be high level-triggered, then the main program would never run because the ISR would be called continuously without the button being pressed, as the hardware is set to pull-up.
 - If we choose the interrupt to be rising-edge triggered, then it would take too long to respond, as it would only trigger after the button is released.
-
2. What is it called when one interrupt preempts another interrupt? What is one condition that must be met for this to occur?
 - When one interrupt preempts another interrupt it is called nested interrupt preemption.
 - For this to occur there should exist an interrupt with higher priority than the one we already have.
 3. A processor is executing and receives a low-priority external interrupt. What factors may impact the latency of handling this interrupt?
 - Factor 1 :- Context switching
 - Factor 2 :- if interrupts are disabled from the main program "critical section" then interrupt has to wait for the critical section to finish.
 - Factor 3 :- If a higher priority interrupt will be called then it has to wait for that higher priority interrupt to finish.

- So we will resume our low priority interrupt execution after the high priority one is finished.
- And ultimately the overall speed of the processor.

4. One issue with nested interrupts is that an ISR may have to be carefully designed to be **reentrant**. A function is reentrant if it can be safely called again while it is in the middle of an execution. You are given the following function:

```
void send_to_radio(char* data) {  
  
    static char data_to_send[10];  
  
    memcpy(data_to_send, data, 10)  
  
    // Assume that this function is reentrant  
  
    radio_send(data_to_send);  
  
}
```

- a. Is the function `send_to_radio` reentrant? Why or why not? **Hint:** Make sure to understand static variables in C.
 - b. What is one simple way to make the function reentrant?
- No, it's not reentrant. Because the `data_to_send[10]` is declared as a static variable hence its store in global memory. This makes it a shared resource and if the function is recalled while `memcpy` is in the middle of execution then, when the function returns back it will start copying the data from which position it left and data will be corrupted.
 - Remove the static keyword which makes the `data_to_send[10]` local so when we resume the execution of the previous call our the data would be the same and not edited by any other program.

Post - Lab

1. A physical action in Lingua Franca, when scheduled, creates an event with a timestamp drawn from the processor's physical clock. This event, once created, has a logical time equal to this timestamp, and any reactions to this event execute at that logical time. Give at least one significant difference here between physical time and logical time in Lingua Franca.
 - A significant difference is that because **physical time** is continuous. In contrast, **logical time** is event based and it increments in discrete steps. Logical time is always behind physical time except for physical action when they are equal.
2. Suppose you are attempting to debug your interrupt handler code, but it is crashing and causing a processor hard fault before exiting the handler. To debug this, you decide to print debug information using `printf` and monitor it on the host computer using screen. But you find that `printf` does not seem to be working.
 - a. What is one reason why `printf` may not work when called from within an ISR?
Hint: The implementation of `printf` is itself using a peripheral device, specifically a device called a UART, and very likely is using interrupts to interact with the device.
 - b. How could you modify either your code or the SDK library code to make `printf` work as expected?
 - c. What are some possible consequences to your modification that could actually make it more difficult to debug the handler code?
 - d. If you decide you do not want to use `printf`, what is one other method by which you could debug your handler? Could this new method have consequences that make it more difficult to debug the handler code?
- a. GP25 pin ISR gets triggered and calls `printf` which fills up the `UARTx_BUFFER` to transfer the string. After UART empties its TX buffer, it triggers a `UARTx_ISR` to request for next character but it is blocked and sent to pending state by the

interrupt controller (NVIC) because our button IO_ISR is running which has higher priority. Now the button ISR cannot end because the UARTx_ISR is in a pending state due to the button ISR. This creates a deadlock.

```

/*! \brief Write to the UART for transmission.
 * \ingroup hardware_uart
 *
 * This function will block until all the data has been sent to the UART transmit buffer
 * hardware. Note: Serial data transmission will continue until the Tx FIFO and
 * the transmit shift register (not programmer-accessible) are empty.
 * To ensure the UART FIFO has been emptied, you can use \ref uart_tx_wait_blocking()
 */
/*
 * \param uart UART instance. \ref uart0 or \ref uart1
 * \param src The bytes to send
 * \param len The number of bytes to send
 */
static inline void uart_write_blocking(uart_inst_t *uart, const uint8_t *src, size_t len) {
    for (size_t i = 0; i < len; ++i) {
        while (!uart_is_writable(uart))
            tight_loop_contents();
        uart_get_hw(uart)->dr = *src++;
    }
}

/*! \brief Read from the UART
 * \ingroup hardware_uart
 *
 * This function blocks until len characters have been read from the UART
 */
/*
 * \param uart UART instance. \ref uart0 or \ref uart1
 * \param dst Buffer to accept received bytes
 * \param len The number of bytes to receive.
 */
static inline void uart_read_blocking(uart_inst_t *uart, uint8_t *dst, size_t len) {
    for (size_t i = 0; i < len; ++i) {
        while (!uart_is_readable(uart))
            tight_loop_contents();
        *dst++ = (uint8_t) uart_get_hw(uart)->dr;
    }
}

```

- b. Instead of using interrupts we continuously check the status register to detect when the Tx_BUFFER is empty and then transfer the next data.
- c. One major drawback of this modification is that pooling the UART data is slow and adds to the latency of the ISR. If this happens the controller will miss any equal or lower priority interrupts.
- d. We can use GPIO pins and LEDs to check whether the ISR has been fired, and if we have an oscilloscope then we can even check how long it took to execute ISR if we toggle the pin at different parts of the code to narrow down at which part it is crashing.
 - But this needs more hardware resources, which we might or might not have.
 - It also needs access to GPIO pins which we might not have access to.

We can also create a hook function which gets triggered when a hardfault is triggered and record the stackframe in a specific location before shutting down or resetting. We can then take a memory dump and analyse the data to figure out where it crashed.

The drawback of this is it requires memory to be reserved for crash logs which might not be available for embedded systems as well as the storage would be limited so only the part of input sequence that can fit into the buffer can only be stored.

3. What were your takeaways from the lab? What did you learn during the lab? Did any results in the lab surprise you?

- We learnt how interrupt preemption works. And what are the consequences of it?
- We learnt what reentrant functions are and how they are useful.
- We learnt how to debug the program with interrupt and which things to take care of
- We learnt how physical actions are used in lingua franca.
- Investigating printf and the deadlock condition was new to us.